

Objektorientiertes Programmieren

Übungsbeispiele

1 Fight Club

In diesem Beispiel implementieren Sie ein rundenbasiertes Kampfsystem für ein textbasiertes Computerspiel, in dem Charaktere abwechselnd gegeneinander antreten. Weiters soll es möglich sein, den Charakteren verschiedene Fähigkeiten zuzuordnen, die dann im Kampf genutzt werden können.



Figure 1: Beispielbild von Chained Echoes. Unten links im Bild werden die unterschiedlichen Skills des aktiven Characters angezeigt. Rechts oben sieht man die Reihenfolge, in welcher die Charaktere an der Reihe sind.

Folgende Punkte müssen dabei beachtet werden:

- Nutzen Sie Vererbung und teilen Sie den Code auf.
- Verwenden Sie sinnvolle Klassen. Beachten Sie dabei die richtige Verwendung von Zugriffsmodifikatoren, Gettern/Settern, den verschiedenen Konstruktoren und Destruktoren sowie der sinnvollen Strukturierung von Funktionen und Daten. Bilden Sie Ihre Klassen in einem entsprechenden Diagramm ab.

- Überprüfen Sie alle Parameterübergaben an Funktionen und Benutzereingaben auf Fehler und verhindern Sie so, dass Ihr Programm bei ungültigen Eingaben nicht mehr richtig funktioniert, Eingaben sollen so lange wiederholt werden, bis sie korrekt sind und der Spielfluss erst dann fortgesetzt werden.
- Testen Sie Ihren Code ausgiebig und berücksichtigen Sie Randbedingungen.

Stufe 1

Zu Beginn soll es möglich sein, aus zumindest zwei vorgefertigten Charakteren zu wählen oder einen eigenen Charakter zu erstellen, jeder Charakter soll dabei zwei Fähigkeiten besitzen. Die Fähigkeiten sollen bei der Charaktererstellung gewählt werden können, implementieren Sie also mehr als zwei Fähigkeiten.

Wurden zwei Charaktere gewählt, sollen diese danach rundenweise unter Verwendung ihrer Fähigkeiten gegeneinander kämpfen. Der Kampf soll so lange andauern, bis einer der Charaktere besiegt wurde.

Gestalten Sie die Regeln für den Kampf selbst.

Nach einem Kampf soll es möglich sein, wieder neue Charaktere zu wählen oder zu erstellen und diese gegeneinander antreten zu lassen.

Weiters soll es eine Möglichkeit geben, sich die Anzahl der Siege und Niederlagen aller Charaktere anzusehen.

Stufe 2

Ermöglichen Sie es Personen, gegen den Computer oder gegeneinander anzutreten und auch Teams zu bilden, die als Klassen implementiert werden. Teams sollen aus mehreren Charakteren bestehen und dann in Team-Kämpfen gegeneinander antreten können. Gestalten Sie auch hier die Regeln selbstständig.

Stufe 3

Erweitern Sie Ihr Spiel um zumindest zwei Entscheidungsbäume als Klassen implementiert, die den Charakteren bei der Erstellung zugeordnet werden können. Diese Bäume sollen das Verhalten der Charaktere im Kampf verändern und sich deutlich voneinander unterscheiden.

Beispiele für Entscheidungsbäume

- Entscheidungsbaum 1: Character heilt Teammitglieder, wenn sie unter 50 Prozent Gesundheit haben. Falls dies nicht der Fall ist, wird dem Gegner Schaden zugefügt.
- Entscheidungsbaum 2: Character fügt dem Gegner mit der meisten Gesundheit Schaden zu.
- Entscheidungsbaum 3: Bei mehr als 2 Gegnern wird ein Gruppenangriff ausgeführt. Bei 2 oder weniger Gegnern wird der Character mit der niedrigeren Gesundheit angegriffen.

Anforderungen

- Fehlerüberprüfung der Inputs
- Erstellung eines UML-Diagramms, welches die Klassenstruktur darstellt
- Stufe 1
 - Implementierung von Fähigkeiten und Charakteren
 - Möglichkeit zur Charaktererstellung
 - Möglichkeit zur Charakterauswahl von bestehenden Charakteren
 - Rundenbasierter Kampf zwischen den Charakteren
 - Protokollierung von Siegen/Niederlagen jedes einzelnen Charakters
 - Ausgabe des Kampfesgeschehens und der Statistiken
- Stufe 2
 - Nutzer können individuelle Teams erstellen
 - Auswahlmöglichkeit zwischen PVP (Player vs. Player) oder PVC (Player vs. Computer)
- Stufe 3
 - Implementierung von mindestens 2 Entscheidungsbäumen
 - Individuelle Zuordnung von Entscheidungsbäumen auf Charaktere

Bewertung

Aspekt	Bewertung
Klassendiagramm	10%
Fehlerprüfung	10%
Stufe 1	30%
Stufe 2	30%
Stufe 3	20%